

ME EN 5960/6960-002: Aerospace CFD

Assignment #1

Due date: Wednesday, September 23rd by 11:59pm

Submission

Assignments can only be submitted on Gradescope, which can be accessed through Canvas. Submissions will be automatically locked at the due date given above.

Use of Generative AI

This assignment expects you to utilize generative AI to progress through it. The goal of this AI use is to help you spend more time on important topics while minimizing time spent on less important/menial tasks throughout the assignment. The decision on what is a less important/menial task lies with the instructor and will be clarified in this section and throughout the assignment. You are responsible for verifying that any AI generated code implements the equations and logic you intended. Remember, we are using AI to help you learn. Please do not use it to skip the learning process. Note that sometimes, you will be learning how to best use AI as a tool for your engineering careers.

As a baseline, you may never:

1. Directly upload this assignment anywhere, including as part of an AI prompt.
2. Copy any part of this assignment and include it as part of an AI prompt. Unless explicitly banned as part of the problem, you may write a prompt in your own words based on part of the assignment.

You may always:

1. Use AI to help you understand what a problem is asking or explain some concept that is required for the problem if you follow all other AI rules. For example, if in the assignment you were asked whether a scheme is explicit/implicit, you may prompt AI with "Please explain the difference between an explicit vs. implicit scheme". You may not prompt it with the scheme you are using and ask whether it is explicit or implicit.
2. Use AI to help you understand how a programming language works, but not how to specifically solve your problem in that language. "How do I write a "for" loop in Python" is fine, "Write a CFD solver for me in Python" is not.

Recommended AI

You may use any AI at your disposal, but I recommend using the University of Utah ChatGPT workspace available to all students for free through <https://ai.utah.edu/tools/chatgpt/get-started.php>. This protects your data and provides a significant amount of free monthly usage. Always monitor your usage limits through settings -> usage, I cannot help you if you run out. At the time of writing, I recommend using ChatGPT 5.6 Sol Medium as it is highly effective but relatively low on resource use. Adjust appropriately as the difficulty of the problem scales.

Programming Language

I *highly* recommend using Python throughout this course, but MATLAB is acceptable if needed.

Assignment

Problem #1: In this problem, you will be solving the one-dimensional linear advection equation for the quantity $u(x, t)$, which bears some light resemblance to the inviscid Navier-Stokes momentum equations.

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0$$

You will solve this equation on the domain $0 \leq x \leq 1$ with constant velocity $a = 1$. Assume that the boundaries are periodic such that $u(0, t) = u(1, t)$. Physically, we expect this equation to transport the initial distribution to the right without changing its shape. See lectures 6 or 7 for examples of the solution.

- a) Conceptual: Assume we want to discretize this equation with an explicit Euler formulation for time. Which of the following three spatial discretization schemes is best to use and why? First-order forward difference, first-order backward difference, or second-order central difference.
- b) Conceptual: If we were solving this equation for $a = -1$, would your choice of discretization change, and how?
- c) Discretize the linear advection equation using an explicit Euler scheme for time, and a first-order backward difference in space. Rearrange your discretized equation to solve for u_i^{n+1} on the LHS and only known variables on the RHS. Your final equation should be written in terms of the CFL number, $CFL = a\Delta t/\Delta x$.
- d) Assume the first point in your spatial domain is $i = 0$ and the last point in your spatial domain is $i = n_x - 1$. Write the mathematical expression that describes the periodic boundary condition as it applies to the quantity u .
- e) Write a program that solves the linear advection using explicit Euler, first-order backward difference by taking advantage of your equation in 1.c, and the boundary conditions in 1.d. Your solver should allow you to easily change 1) the number of grid points n_x , 2) the CFL number, 3) and the final simulation time t_{end} . Use a smooth initial condition given by $u(x, 0) = \exp[-100(x - 0.5)^2]$. Your initial simulation should use $n_x = 100$ and $CFL = 0.8$ and run until $t_{end} = 1$. **AI USAGE:** You may not use AI to write any code for this problem. However, you can use AI as an assistant to decide on a code structure or help you understand how/why to do one specific component of the problem at a time. You are welcome to use AI to help understand how to install/run the code in Python if needed.

A simple structure for your code is given below, but feel free to do whatever works best for you:

- a. Declare all variables and constants
- b. Calculate any intermediate quantities like Δx and Δt
- c. Create a one-dimensional array named, for example, "x" with n_x entries for your grid. Each entry x_i should correspond to the spatial coordinate at that grid point. Note that x_0 should correspond to $x = 0$ and x_{n_x-1} should correspond to $x = 1$.

- d. Create a one-dimensional array named, for example, “u_n” with n_x entries. This will house your solution at the current time. Each entry u_i should correspond to the value of u at that grid point at the current time and the positions should line up with your “x” array. Fill this array with the initial condition given above using the “x” array.
 - e. Create a one-dimensional array named, for example, “u_np1” with n_x entries. This will house your solution at the next time. You may fill this with zeroes for the moment.
 - f. Create two-nested “for” loops. Iterate over n outside (your time index) and iterate over i inside (your space index). For time, your loop should run from $0 \leq t \leq t_{end}$, and for space, your loop should run from $0 \leq x \leq 1$. Note the distinction between the indices i, n and the actual coordinate/time x, t .
 - g. As you go through one full spatial loop, you should use your discretization equation to update the value of “u_np1” at every point. When necessary, use boundary conditions to fill in missing information through “if” statements. This is your updated solution.
 - h. At this point, you may want to save the values in one or both of your “u” arrays. This will allow you to create animations/perform analyses on the data as it changes over time.
 - i. Before the time index iterates, set “u_n” = “u_np1” as we are moving on to the next timestep and the “new solution” is now the “current solution”. The solution of “u_np1” is now ready to be overwritten.
 - j. The code should now iterate in time and calculate a new set of “u_np1” values as it iterates in space. Continue this until you have reached t_{end} . This requires tracking Δt .
 - k. Congratulations, you have written your first CFD code! Feel free to adjust and improve.
- f) Conceptual: What *should* the final solution at $t_{end} = 1$ look like? Think about how the linear advection equation simply transports the initial distribution to the right without changing its shape at speed a . Consider the length of the domain and the periodic boundary conditions. Does your final solution (i.e., “u_np1” plotted as a function of “x” at the end of the last iteration) resemble your expectations? What differences can you note?
- g) Plot the initial and final numerical solutions (u vs. x) on the same figure. Note any differences.
- h) Add features to your code that allow you to save the u array at each time and output u vs. x as an animation that evolves over time. Rerun your code and describe what happens in the video. **AI USAGE**: You may prompt the AI to fully develop this feature for you. However, you may only do this without copying any of your code into the prompt. Instead, your prompt will likely need to supply some information about the structure of the code (e.g., where do you store variables? How are they updated?) so the AI can provide the required functionality.
- i) Using $n_x = 100$ and $t_{end} = 2$, repeat the calculation with $CFL = [0.9, 1.0, 1.1]$. Note that these are three different simulations at different CFL numbers. Before you run these cases, classify each as “stable” or “unstable”. At what CFL number does the numerical behavior change? What happens when stability restrictions are violated? What features tell you a solution is unstable? **AI**

USAGE: You may use AI as an agent to run these three simulations, or have it add logic to your code to run all three at once and output the solutions.

- j) Conceptual: What is the downside of running at very small CFL numbers just to ensure stability?
- k) Perform the calculation with the parameters given in 1.e but use the following four values of $n_x = [50, 100, 200, 400]$. The CFL number should be kept constant at $CFL = 0.8$. (Note, again, these are four different simulations you are being asked to run). Plot all four of the final solutions together with the initial solution. Add a feature to your code that approximates the amount of time it takes the code to run. **AI USAGE:** You may use AI as an agent to run these simulations, or have it add logic to your code to run all of them at once and output the solutions. AI can develop the timing logic for your code.
- l) Plot the amount of time the simulation takes to run as a function of the number of points. Mathematically, what kind of trend do you expect and why (consider how cost scales with number of points and number of timesteps)? Does it match what you see?
- m) How does refining the grid change the result?
- n) Since we know the final solution *should* exactly match the initial solution (for the given advective velocity, domain size, and final time), we can calculate the error of any solution as:

$$\epsilon = \frac{1}{n_x} \sum_i |u_{i,\text{final}} - u_{i,\text{initial}}|$$

That is, at each point, subtract the absolute value of the initial solution from the final solution, sum over all points, and divide by the total number of points. Calculate this for each of the four grid resolutions above and plot as ϵ vs. Δx (note, Δx not n_x). What happens to error as $\Delta x \rightarrow 0$? What kind of slope do we expect for the error as a function of grid size in a first order scheme? Does your slope match the expectation?

Problem #2: In this problem, you will be solving the one-dimensional inviscid Burgers equation for the quantity $u(x, t)$, which bears even more resemblance to the compressible Navier-Stokes equation, especially due to the non-linear velocity term that now appears.

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = 0$$

You will solve this equation on the domain $0 \leq x \leq 1$. Assume that the boundaries are periodic such that $u(0, t) = u(1, t)$. The initial condition can again be assumed as $u(x, 0) = \exp[-100(x - 0.5)^2]$. Note that $u > 0$ everywhere for this problem, but this is not strictly required and can change over time/space (thereby possibly requiring different discretization).

- Conceptual: The linear advection equation advected a pulse at velocity a . Based on visual similarities between that equation and Burgers equation, what velocity do you expect quantities to be advected for Burgers equation? Looking at the initial conditions, identify which portions of the wave you expect to travel fastest and slowest. Do you expect translation to occur with or without the pulse shape changing?
- Conceptual: Sketch what you expect your solution to look like after some amount of time relative to the initial condition. (Non-trivial but doable if you really think hard)
- Consider the following two discrete forms of Burgers equation:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + u_i^n \left(\frac{u_i^n - u_{i-1}^n}{\Delta x} \right) = 0$$

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + u_i^{n+1} \left(\frac{u_i^{n+1} - u_{i-1}^{n+1}}{\Delta x} \right) = 0$$

Derive the update formula for each case with $n + 1$ terms on the left and n terms on the right. Describe the difference between the two, and comments on how the solution strategy would differ between the two.

- Write a program that solves the Burgers equation using the first discrete formulation. You should start with your previous code and adjust where necessary for this new problem (e.g., replacing the previous discretized update equation with the new one). Your final solver should allow you to easily change 1) the number of grid points n_x , 2) the CFL number, 3) and the final simulation time t_{max} . Your initial simulation should use $n_x = 100$ and $CFL = 0.8$ and run until $t_{end} = 1$. Note that because your advective velocity is no longer constant, you will have to dynamically compute $CFL = u\Delta t/\Delta x$ at each point and pick the most restrictive condition to use as your timestep. You will additionally need to update Δt each timestep. **AI USAGE:** You may not use AI to write any code for this problem. However, you can use AI as an assistant to decide on a code structure or help you understand how/why to do one specific component of the problem at a time.

- e) Generate a video of your simulation over time. Describe what happens. Specifically note what happens to the magnitude of gradients in your simulation.

- f) Consider the conservative form of Burgers equation by noting $d\left(\frac{u^2}{2}\right) = udu$:

$$\frac{\partial u}{\partial t} + \frac{\partial u^2/2}{\partial x} = 0$$

Mathematically, this equation is identical to what we just studied, but the conservative form is relevant for when we make numerical approximations. Note that while the PDE is identical, the *approximation* does not necessarily have to be identical which leads to differences. We can discretize it with explicit Euler and first order backward difference as:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + \left(\frac{(u_i^n)^2 - (u_{i-1}^n)^2}{2\Delta x} \right) = 0$$

- g) Write a program that solves the conservative form of the Burgers equation using the discretization given above. **AI USAGE:** AI may fully write this code for you but remember that you should not directly copy any part of this document for your prompt.
- h) Generate one figure of u vs. x at t_{end} that includes both the conservative form and the non-conservative form solution. How do they compare? Can you tell whether one solution is more correct than another?
- i) Let us track the total amount of “ u ” in the domain. Since there is no source/sink in the equation, the total quantity should be conserved. Define:

$$Q(t) = \sum_i u_i(t)\Delta x$$

as the amount of “ u ” in the entire domain at any time. Calculate this quantity for both the conservative and non-conservative forms of the Burgers equation and plot them together on one figure as a function of time. Additionally, plot a horizontal line representing the initial value $Q(0)$. Comments on the differences between the two forms of the equation.